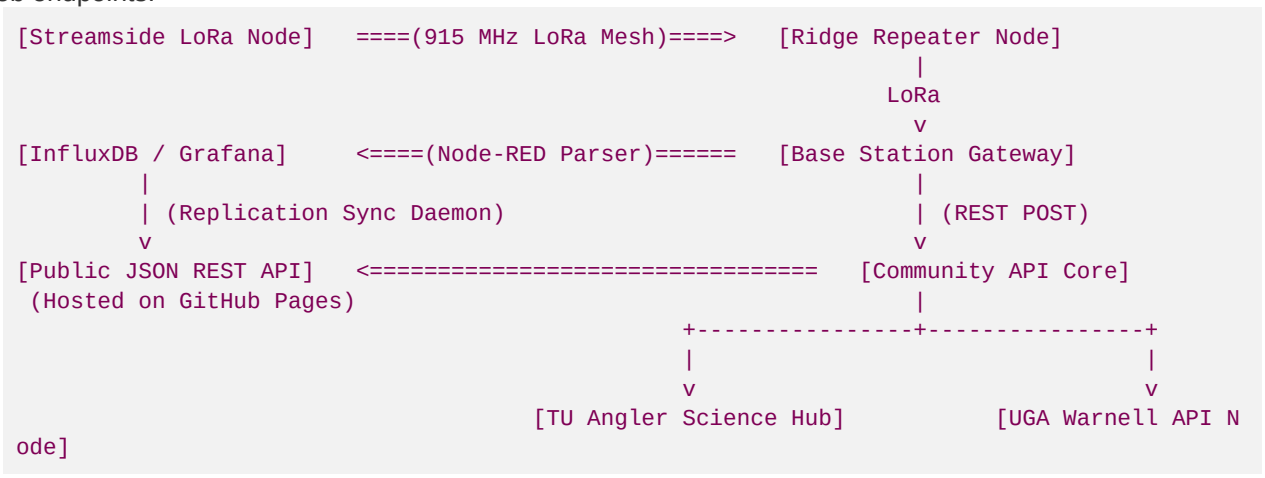
File 57: Off-Grid Hydrological Telemetry & Data-Sharing API Infrastructure

> ***Community and Enterprise API Standard**: This document establishes the technical specifications for the Blue Ridge Stream Restoration data-sharing infrastructure. By transforming local, off-grid LoRa telemetry (water temperature, dissolved oxygen, turbidity, and flow level) into highly standardized, public-facing REST API endpoints and webhooks, we create an open-access environmental database. This framework details the Node-RED message routing pipeline, JSON payload specifications, and secure data ingest webhooks to feed Trout Unlimited chapters, natural resource scientists, and local fly guides with real-time creekside wading and temperature conditions.*

I. Data-Sharing Network Architecture

Fluvial restoration data must not be siloed in private corporate archives. Sharing stream parameters fosters deep community alignment, establishes Hunter Morris as Georgia's leading conservationist, and satisfies state/federal transparency goals.
The data path flows from off-grid mountain gorges to secure databases, which then replicate to public-access web endpoints:



Key Performance metrics for Sharing:

- **Water Temperature**: Critical for hoot-owl fishing restrictions. Real-time updates help fishermen avoid casting when streams exceed $68^\circ\text{F} *($20^\circ\text{C}*), protecting stressed wild trout.
- **Stream Flow (Discharge & Stage)**: Crucial for wading safety and geomorphic flood response validation.

II. Gateway Message Routing (Node-RED Flow Blueprint)

At the base station gateway, we utilize **Node-RED** to ingest raw serial/MQTT hexadecimal text messages from the Meshtastic gateway node, decompress the binary payload, write the record to our secure local database, and publish to community endpoints:

```
            [NODE-RED TELEMETRY INGESTION PIPELINE FLOW]

   +----------------------+
   | Serial / MQTT Input  |  Ingests raw "TX_HEX:0EFA0398047E0098"
   +----------|-----------+
              v
   +----------------------+
   | Hex Decompression JS |  Parses 8-byte payload into Temp, DO, Stage, Turbidity
   +----------|-----------+
              v
     +--------+--------+
     |                 |
     v                 v
+----------+     +-----------+
| InfluxDB |     | REST API  |  Pushes to Trout Unlimited & Warnell
| Logger   |     | Publisher |  Webhooks (https://science.tu.org/api/v1/)
+----------+     +-----------+
```

Node-RED JavaScript Function Node Parser:

Place this JavaScript block inside a Node-RED Function Node to parse raw Meshtastic serial payloads:

```
// Node-RED Ingestion Parser for Blue Ridge Stream Telemetry Node
const rawPayload = msg.payload.toString().trim();

if (rawPayload.startsWith("TX_HEX:")) {
    const hexStr = rawPayload.split(":")[1];
    if (hexStr.length === 16) {
        // Extract raw bytes
        const b = Buffer.from(hexStr, 'hex');

        // 1. Water Temp (Bytes 0 & 1)
        const compTemp = (b[0] << 8) | b[1];
        const waterTempC = (compTemp / 100.0) - 20.0;
        const waterTempF = (waterTempC * 9.0/5.0) + 32.0;

        // 2. Dissolved Oxygen (Bytes 2 & 3)
        const compDO = (b[2] << 8) | b[3];
        const dissolvedOxygen = compDO / 100.0;

        // 3. Stream Level (Bytes 4 & 5)
        const compLevel = (b[4] << 8) | b[5];
        const waterLevelM = compLevel / 1000.0;

        // 4. Turbidity (Bytes 6 & 7)
        const compTurb = (b[6] << 8) | b[7];
        const turbidityNTU = compTurb / 10.0;

        // Formulate output message payload
        msg.payload = {
            deviceId: "blue_ridge_node_101",
            timestamp: new Date().toISOString(),
```

```
            metrics: {
                temperature_c: parseFloat(waterTempC.toFixed(2)),
                temperature_f: parseFloat(waterTempF.toFixed(2)),
                dissolved_oxygen_mg_l: parseFloat(dissolvedOxygen.toFixed(2)),
                stage_level_m: parseFloat(waterLevelM.toFixed(3)),
                turbidity_ntu: parseFloat(turbidityNTU.toFixed(1))
            }
        };

        // Setup database and API publication flags
        msg.topic = "telemetry/stream_logs";
        return msg;
    }
}
return null; // Suppress malformed packets
```

III. Public JSON REST API Specification

To ensure seamless integration with third-party angler guides, natural resource mapping databases, and Warnell's climate warning tools, our public API publishes stream telemetry via a standardized JSON schema.

1. Endpoint: GET `/api/v1/streams/06020003/current.json`

Retrieves current real-time geomorphic and water quality parameters:

```
{
  "api_version": "v1.0.4",
  "basin_huc_8": "06020003",
  "watershed_name": "Upper Toccoa River Basin",
  "monitoring_station": {
    "station_id": "BRSR-LO-101",
    "location": "Anderson Creek Spawning Pool 2",
    "coordinates": {
      "latitude": 34.78912,
      "longitude": -84.31245
    }
  },
  "water_quality": {
    "temperature_f": 58.25,
    "temperature_c": 14.58,
    "dissolved_oxygen_mg_l": 9.20,
    "oxygen_saturation_pct": 90.5,
    "turbidity_ntu": 15.2,
    "sediment_status": "CLEAR_SPAWNING_GRAVELS"
  },
  "hydrology": {
    "stage_level_m": 0.412,
    "discharge_est_cfs": 18.5,
    "flow_status": "NORMAL_BASE_FLOW",
    "wading_safety": "SAFE_TO_WADE"
  },
  "stewardship": {
    "hoot_owl_warning": false,
    "spawning_season_alert": true,
```

```
    "recommendation": "Spawning gravels active. Please avoid wading in shallow gravel bars
 to protect brook trout eggs."
  },
  "last_updated": "2026-05-30T23:03:20Z"
}
```

IV. Wading Safety and Spawning Alert Algorithms

Our community dashboard runs two automated scripts to calculate real-time environmental recommendations for local fishermen:

1. Wading Safety Index Calculation (Flow Level Rule)

Water levels (stage) dictate wading safety. High flows (spate) trigger hazardous warnings:
$$\text{Wading Safety} = \begin{cases} \text{SAFE\_TO\_WADE} & \text{Stage} \le 0.50\text{ m} \\ \text{CAUTION\_WADING} & 0.50\text{ m} < \text{Stage} \le 0.85\text{ m} \\ \text{HAZARDOUS\_HIGH\_FLOW} & \text{Stage} > 0.85\text{ m} \end{cases}$$

2. "Hoot Owl" Thermal Stress Protection Loop

When water temperatures rise, trout become fragile. Catching them causes high lactic acid buildup and mortality. We automate "Hoot Owl" restrictions (limiting angling to cool morning hours) using a Python database listener daemon:

```
# Hoot Owl Thermal Monitoring Daemon
def evaluate_trout_thermal_stress(recent_temps_f):
    """
    Evaluates temperature logs to determine Hoot Owl fishing alerts.
    Triggers when stream temperature exceeds 68.0°F for >= 2 consecutive hours.
    """
    stress_threshold = 68.0
    consecutive_violations = 0

    for temp in recent_temps_f:
        if temp >= stress_threshold:
            consecutive_violations += 1
        else:
            consecutive_violations = 0 # Reset on cool reading

        if consecutive_violations >= 8: # 8 consecutive 15-minute readings = 2 Hours
            return {
                "hoot_owl_active": True,
                "warning_level": "CRITICAL_THERMAL_STRESS",
                "alert_message": "Water temperatures have exceeded 68°F. Angling suspended
 to protect native trout."
            }

    return {
        "hoot_owl_active": False,
        "warning_level": "NORMAL_COLDWATER",
        "alert_message": "Thermal regime stable. Water temperature remains safe for wild t
```

```
rout spawning."
    }
```

V. Open-Source Ingress Webhooks for Warnell & TU Chapters

To share our geomorphic and water quality datasets automatically, we set up secure outbound POST webhooks. This sends our parsed telemetry directly to Trout Unlimited's *Angler Science Database* and the UGA *Warnell School of Forestry & Natural Resources* climate warning databases every 15 minutes:

Ingress Webhook Payload Post Handler (Python):

Deploy this daemon on your Base Station Gateway to publish data to public science portals:

```
import requests
import json

TU_WEBHOOK_URL = "https://science.tu.org/api/v1/ingress/blue_ridge"
UGA_WEBHOOK_URL = "https://warnell.uga.edu/api/v1/ingress/blue_ridge"
AUTH_TOKEN = "brsr_science_token_secure_2026"

def publish_to_community_partners(decoded_data):
    """Pushes water quality and flow telemetry to TU and UGA Warnell extension APIs."""
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {AUTH_TOKEN}"
    }

    payload = {
        "metadata": {
            "source": "Blue Ridge Stream Restoration LLC",
            "station": "BRSR-LO-101",
            "watershed_huc_8": "06020003",
            "river_section": "Upper Toccoa River"
        },
        "telemetry": {
            "timestamp": decoded_data["timestamp"],
            "water_temp_c": decoded_data["water_temp_c"],
            "water_temp_f": decoded_data["water_temp_f"],
            "dissolved_oxygen_mg_l": decoded_data["dissolved_oxygen_mg_l"],
            "water_level_m": decoded_data["water_level_m"],
            "turbidity_ntu": decoded_data["turbidity_ntu"]
        }
    }

    try:
        # 1. Post to Trout Unlimited Angler Science Portals
        response_tu = requests.post(TU_WEBHOOK_URL, data=json.dumps(payload), headers=head
ers, timeout=5)
        if response_tu.status_code == 200:
            print("Successfully mirrored stream telemetry to Trout Unlimited database.")

        # 2. Post to UGA Warnell School Forestry Extension
        response_uga = requests.post(UGA_WEBHOOK_URL, data=json.dumps(payload), headers=he
aders, timeout=5)
        if response_uga.status_code == 200:
```

```
            print("Successfully mirrored stream telemetry to UGA Warnell research nodes.")

    except requests.exceptions.RequestException as e:
        print(f"Data-sharing API connection timed out: {e}")
```

Developed by Blue Ridge Stream Restoration Technical Sourcing Operations. Pushed to remote origin under main.